

""" Erzeugt eine 4x4 Matrix als Eingabebild (Greyscale)
 - Mockup. Ein Pixel nahe 0 wird als Schwarz angezeigt;
 ein Pixel nahe 255 wird als Weiss angezeigt.

0 - 255
 ↗ ↖
 Schwarz Weiss

"""

```
input-matrix = np.array([
    [200, 10, 30, 40],
    [10, 200, 10, 10],
    [10, 10, 110, 120],
    [130, 140, 150, 160]
])
```

""" Definiert einen 3x3 Filter-kernel (Hier: Kantenerkennung)
 Die zentrale '8' umgeben von '-1', hebt abrupte
 Helligkeitsunterschiede hervor """

```
kernel = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
])
```

```
def apply_convolution(matrix, kernel):
```

Dimensionen ermitteln: m = Zeilen, n = Spalten

m, n = matrix.shape

km, kn = kernel.shape

Berechnen die Ausgabegröße: (Eingabe - kernel + 1)
 # Gilt für "valid" padding

output = np.zeros((m - km + 1, n - kn + 1))

```
# Äussere Schleifen schieben den Filter über das Bild  
# (Schrittweite = 1)
```

```
for i in range(output.shape[0]):
```

```
    for j in range(output.shape[1]):
```

```
        # Slicing: Extrahiert das aktuelle 3x3 Fenster  
        # aus der Matrix
```

```
        region = Matrix[i:i+km, j:j+kn]
```

```
        # Elementweise Multiplikation gefolgt von der  
        # Summenbildung  
        # Math: Value = sum(region_ij * kernel_ij)
```

Bsp.

$$\rightarrow \sum (A \odot B)$$

Hadamard
Produkt

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \odot \begin{pmatrix} e & f \\ g & h \end{pmatrix}$$

$$\begin{pmatrix} a \cdot e & b \cdot f \\ c \cdot g & d \cdot h \end{pmatrix}$$

$$\text{Convolution} \Rightarrow a \cdot e + b \cdot f + c \cdot g + d \cdot h$$

```
value = np.sum(region * kernel)
```

```
# Speichert das Ergebnis in der Ausgabe  
# Feature-map
```

```
output[i, j] = value
```

```
return output
```

⇒ Mathematischer Verbindung

* Bei einer "Valid" convolution (ohne Padding) verringert sich der dimensionen der Ausgangsmatrix, da der Filter nicht über den Rand hinausragen darf.

Parameter:

W_{in} , H_{in}

Eingabebreite - und -höhe

K_w , K_h

Filterbreite - und -höhe

S

Schrittweite (in diesem Code $S = 1$)

Allgemeine Formel - Ausgangsmatrix Dimensionen

$$W_{out} = \frac{W_{in} - K_w}{S} + 1$$

$$H_{out} = \frac{H_{in} - K_h}{S} + 1$$

Rechenbeispiel:

Für unsere 4×4 Eingabematrix und den 3×3 Filter mit einer Schrittweite von 1:

$$\text{Output Breite} = \frac{4 - 3}{1} + 1 = 2$$

$$\text{Output Höhe} = \frac{4 - 3}{1} + 1 = 2$$

$\Rightarrow$ Die resultierenden Feature Map ist eine 2×2 Matrix.

Convolution Logic:

Der Ausgangswert $y_{i,j}$ ist die Summe der elementweise Produkte:

$$y_{i,j} = \sum_{m=0}^{K_w-1} \sum_{n=0}^{K_h-1} x_{i+m,j+n} \cdot K_{m,n}$$

$$= \sum (x \odot k)$$

import cv2 # cv2 ist openCV => Bildverarbeitung

Kantenerkennung

```
kernel = np.array([  
    [-1, -1, -1],  
    [-1, 8, -1],  
    [-1, -1, -1]  
])
```

Identitätskern für Farberkennung

```
red_kernel = np.array([  
    [0, 0, 0],  
    [0, 1, 0],  
    [0, 0, 0]  
])
```

```
blue_kernel = red_kernel.copy()
```

Mockup RGB-Matrix (4 x 4 x 3 Tensor)

```
mock_image = np.array([  
    [[255, 0, 0], [0, 255, 0] ...  
    ...  
    [255, 255, 255]]  
], dtype = np.uint8)
```

```
def convolve_channel(channel, kernel):
```

Nutzt die zuvor definierte apply_convolution()

```
    return apply_convolution(channel, kernel)
```

English: This is a wrapper-function!

Extraktion einzelne Kanäle (slicing)

```
red_channel = mock_image[:, :, 0]
```

```
green_channel = mock_image[:, :, 1]
```

```
blue_channel = mock_image[:, :, 2]
```

Spezifische Farben erkennen

red_detected = convolve_channel(red_channel, red_kernel)

blue_detected = convolve_channel(blue_channel, blue_kernel)

Kantenerkennung benötigt 2D-Intensität

grey_image = cv2.cvtColor(mock_image, cv2.COLOR_RGB2GRAY)

edge_detected = apply_convolution(grey_image, edge_kernel)

Mathematische Verbindung

Ein Farbbild ist eine 3D Tensor der Form (H, W, C) .

⇒ Slicing erlaubt die Isolation spezifischer Farben!

Bsp: $\text{image}[i, :, 0] \rightarrow \text{Rot-Intensität}$

Grayscale conversion (Graustufenumwandlung):

⇒ Um Kantenerkennung über alle Farben anzuwenden, wandeln wir RGB in einem Intensitätswert Y um:

$$Y = 0.299R + 0.587G + 0.114B$$

$$Y \approx \text{cv2.cvtColor}(\text{mock_image}, \text{cv2.COLOR_RGB2GRAY})$$

Farbisolation:]

⇒ Die Verwendung eines Identitätskerns auf einem Kanal bewahrt dessen Werte, während die Dimensionen gemäß die Formel reduziert werden:

$$\text{Output Size} = (W_{in} - K_w + 1) \times (H_{in} - K_h + 1)$$

Slicing und Tensor-Projektion:

In Python nutzen wir die Syntax:

`mock_image[Zeilen, Spalten, Kanal],`

um gezielten Daten zu extrahieren. Das Slicing der dritten Dimension (Index 0) "Projiziert" den 3D Farbwürfel auf eine 2D Identitätskarte.

Bsp. Wenn wir

`red-channel = mock_image[:, :, 0]`

ausführen, extrahieren wir nur den ersten Wert jedes RGB Triplets.

Beispiel: RGB Triplet `[255, 255, 0]` $\Rightarrow$ GEIB
Slice: Index 0 $\Rightarrow$ 255 Rot-Anteil

Für unser Roter-Kanal haben wir:

$$\text{red-channel} = \begin{bmatrix} 255 & 0 & 0 & 255 \\ 255 & 255 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 255 & 255 & 0 & 255 \end{bmatrix}$$

Warum das wichtig ist in der KI:

In der KI betrachtet das CNN (Convolutional Neural Network) jeden Farbkanal in der ersten Ebene als separate "Feature Map".

Durch die isolierte Betrachtung des Rot-Kanals kann die KI Objekte finden, die nur im roten Licht sichtbar sind - entscheidend für Aufgaben wie die Erkennung von Stopp-Schilden.

PCA (Hauptkomponenten Analyse)

```
def apply_pca(image):
```

```
# Schritt 1: Zentrierung (Mittelwert jede Spalte abziehen)
```

```
mean_vec = np.mean(image, axis=0)  
centered_data = image - mean_vec
```

```
# Schritt 2: Kovarianzmatrix A (Spalten als Variabeln)
```

```
#  $A_{ij} = 1/(n-1) * \sum (x_i - \bar{x}_i)(x_j - \bar{x}_j)$ 
```

```
cov_matrix = np.cov(centered_data, rowvar=False)
```

```
# Schritt 3: Eigenwertzerlegung (Charakteristische Gleichung)
```

```
eig_vals, eig_vecs = np.linalg.eigh(cov_matrix)
```

```
# Schritt 4: Sortierung (Höchste Eigenwerte (Varianz) zuerst)
```

```
sorted_idx = np.argsort(eig_vals)[::-1]
```

```
eig_vals_sorted = eig_vals[sorted_idx]
```

```
eig_vecs_sorted = eig_vecs[:, sorted_idx]
```

```
# Schritt 5: Erklärte Varianz
```

```
explained_variance_ratio = eig_vals_sorted/np.sum(eig_vals_sorted)
```

```
cumulative_variance = np.cumsum(explained_variance_ratio)
```

```
return centered_data, mean_vec, eigvecs_sorted,  
        explained_variance_ratio, cumulative_variance
```